

Introduction to Pandas:

Pandas is a software library written for the Python programming language.

It has functions for analyzing, cleaning, exploring, and manipulating data.

The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis"

Pandas is usually imported under the pd alias.

```
In [99]: import pandas as pd
```

Key Data Structures:

1. series

A Pandas Series is like a column in a table.

It is a one-dimensional array holding data of any type.

It can be created from lists, arrays, or dictionaries.

Each element in a Series is associated with an index label.

```
In [100... import pandas as pd

a = [1, 7, 2]

myvar = pd.Series(a)

print(myvar)
```

```
0    1
1    7
2    2
dtype: int64
```

Labels

If nothing else is specified, the values are labeled with their index number. First value has index 0, second value has index 1 etc.

This label can be used to access a specified value.

```
In [102... print(myvar[1])
```

```
7
```

```
In [4]: ### create labels

import pandas as pd

a = [1, 7, 2]

myvar = pd.Series(a, index = ["x", "y", "z"])

print(myvar)

x    1
y    7
z    2
dtype: int64
```

```
In [5]: print(myvar["x"])

1
```

from a dictionary

```
In [6]: import pandas as pd

calories = {"day1": 420, "day2": 380, "day3": 390}

myvar = pd.Series(calories)

print(myvar)

day1    420
day2    380
day3    390
dtype: int64
```

```
In [7]: myvar["day1"]

Out[7]: 420
```

also we can specify

```
In [8]: import pandas as pd

calories = {"day1": 420, "day2": 380, "day3": 390}

myvar = pd.Series(calories, index = ["day1", "day2"])

print(myvar)

day1    420
day2    380
dtype: int64
```

2. DataFrames

Data sets in Pandas are usually multi-dimensional tables, called DataFrames.

Series is like a column, a DataFrame is the whole table.

Each column in a DataFrame is a Series.

```
In [110...] import pandas as pd

data = {"names": ["Amal", "Binu", "Charles"], "age": [50, 40, 45]}

df = pd.DataFrame(data)

print(hai)

df
```

```
   names  age
0   Amal   50
1   Binu   40
2 Charles   45
```

```
Out[110]:
```

	names	age
0	Amal	50
1	Binu	40
2	Charles	45

Locate Row

As you can see from the result above, the DataFrame is like a table with rows and columns.

Pandas use the loc attribute to return one or more specified row(s)

```
In [115...] print(df)
df.loc[[0]]
```

```
   names  age
0   Amal   50
1   Binu   40
2 Charles   45
```

```
Out[115]:
```

	names	age
0	Amal	50

```
In [116...] print(df.loc[[0, 2, 1]])
```

```
   names  age
0   Amal   50
2 Charles   45
1   Binu   40
```

Named Indexes

With the index argument, you can name your own indexes.

```
In [117...] import pandas as pd

data = {
    "calories": [420, 380, 390],
```

```

    "duration": [50, 40, 45]
}

df = pd.DataFrame(data, index = ["day1", "day2", "day3"])

print(df)

```

	calories	duration
day1	420	50
day2	380	40
day3	390	45

In [118... *#refer to the named index:*

```
print(df.loc[["day2", "day1", "day3"]])
```

	calories	duration
day2	380	40
day1	420	50
day3	390	45

In [119... df

Out[119]:

	calories	duration
day1	420	50
day2	380	40
day3	390	45

access columns

In [121... df[["calories"]]

Out[121]:

	calories
day1	420
day2	380
day3	390

In [124... df[["calories", "duration"]]

Out[124]:

	calories	duration
day1	420	50
day2	380	40
day3	390	45

Reading Data:

Read data from various sources using functions like `pd.read_csv()`, `pd.read_excel()`

In [128...

```
import pandas as pd

data = pd.read_csv('C:/Users/HP/medical_insurance_data.csv')
data
```

Out[128]:

	Age	Sex	BMI	Children	Smoker	Region	Occupation	Exercise_Frequency	Chronic_Cor
0	56.0	Male	26.787631	0.0	No	East	White Collar	Daily	
1	69.0	Male	22.063653	4.0	Yes	West	White Collar	Monthly	
2	46.0	Female	24.036101	2.0	Yes	East	White Collar	Rarely	
3	32.0	Male	24.229816	2.0	No	North	Blue Collar	Rarely	
4	60.0	Male	31.784284	0.0	NaN	East	Blue Collar	Weekly	
...	...	...	...	...	...	...	...	...	...
995	18.0	Male	17.794337	0.0	Yes	West	Other	Monthly	
996	35.0	Female	22.538245	1.0	Yes	NaN	Other	Monthly	
997	49.0	NaN	18.285478	4.0	No	South	Other	Monthly	
998	64.0	Male	22.096111	2.0	No	West	White Collar	Rarely	
999	66.0	Female	NaN	2.0	No	South	Blue Collar	Rarely	

1000 rows × 35 columns



In [178...

```
data.iloc[0:10,0:4]
```

Out[178]:

	Age	Sex	BMI	Children
0	56.0	Male	26.787631	0.0
1	69.0	Male	22.063653	4.0
2	46.0	Female	24.036101	2.0
3	32.0	Male	24.229816	2.0
4	60.0	Male	31.784284	0.0
5	25.0	Female	23.202814	4.0
6	78.0	Male	31.783167	2.0
7	38.0	Male	11.437548	2.0
8	56.0	Male	31.739279	2.0
9	75.0	Male	NaN	4.0

In [127...

```
x= pd.read_csv("C:/Users/HP/Desktop/datasets/pandas1.csv")
x
```

Out[127]:

	name	age	gender
0	a	12	m
1	a	12	f
2	a	12	m
3	a	12	f
4	a	12	m
5	b	13	f
6	b	13	m
7	b	13	f
8	b	13	m
9	b	13	f
10	b	13	m
11	b	13	f
12	b	13	m
13	b	13	f
14	b	13	m
15	b	13	f
16	b	13	m
17	b	13	f
18	b	13	m
19	b	13	f
20	b	13	m
21	b	13	m
22	b	13	f
23	b	13	m
24	b	13	f
25	b	13	m
26	b	13	f
27	b	13	m
28	b	13	f
29	b	13	m
30	b	13	f
31	b	13	m
32	b	13	f
33	b	13	m

	name	age	gender
34	b	13	f

viewing Data:

Inspect the first few rows with head(), last few rows with tail()

In [129...]

data.head() #5

Out[129]:

	Age	Sex	BMI	Children	Smoker	Region	Occupation	Exercise_Frequency	Chronic_Condi
0	56.0	Male	26.787631	0.0	No	East	White Collar	Daily	
1	69.0	Male	22.063653	4.0	Yes	West	White Collar	Monthly	
2	46.0	Female	24.036101	2.0	Yes	East	White Collar	Rarely	
3	32.0	Male	24.229816	2.0	No	North	Blue Collar	Rarely	
4	60.0	Male	31.784284	0.0	NaN	East	Blue Collar	Weekly	

5 rows × 35 columns



In [18]:

data.head(10)

Out[18]:

	Age	Sex	BMI	Children	Smoker	Region	Occupation	Exercise_Frequency	Chronic_Condi
0	56.0	Male	26.787631	0.0	No	East	White Collar	Daily	
1	69.0	Male	22.063653	4.0	Yes	West	White Collar	Monthly	
2	46.0	Female	24.036101	2.0	Yes	East	White Collar	Rarely	
3	32.0	Male	24.229816	2.0	No	North	Blue Collar	Rarely	
4	60.0	Male	31.784284	0.0	NaN	East	Blue Collar	Weekly	
5	25.0	Female	23.202814	4.0	No	West	Other	NaN	
6	78.0	Male	31.783167	2.0	No	South	White Collar	NaN	
7	38.0	Male	11.437548	2.0	No	West	Blue Collar	Weekly	
8	56.0	Male	31.739279	2.0	Yes	East	White Collar	Monthly	
9	75.0	Male	NaN	4.0	Yes	East	Blue Collar	Weekly	

10 rows × 35 columns

In [19]: `data.tail()#5`

Out[19]:

	Age	Sex	BMI	Children	Smoker	Region	Occupation	Exercise_Frequency	Chronic_Cor
995	18.0	Male	17.794337	0.0	Yes	West	Other	Monthly	
996	35.0	Female	22.538245	1.0	Yes	NaN	Other	Monthly	
997	49.0	NaN	18.285478	4.0	No	South	Other	Monthly	
998	64.0	Male	22.096111	2.0	No	West	White Collar	Rarely	
999	66.0	Female	NaN	2.0	No	South	Blue Collar	Rarely	

5 rows × 35 columns

In [20]: `data.tail(10)`

Out[20]:	Age	Sex	BMI	Children	Smoker	Region	Occupation	Exercise_Frequency	Chronic_Cor
990	22.0	Female	30.795076	4.0	Yes	South	NaN	Rarely	
991	72.0	Male	25.605590	3.0	Yes	West	White Collar	Rarely	
992	73.0	Male	22.575165	0.0	No	South	White Collar	Monthly	
993	50.0	Male	28.877227	NaN	Yes	West	White Collar	Weekly	
994	71.0	Female	13.078217	1.0	No	NaN	Blue Collar	Monthly	
995	18.0	Male	17.794337	0.0	Yes	West	Other	Monthly	
996	35.0	Female	22.538245	1.0	Yes	NaN	Other	Monthly	
997	49.0	NaN	18.285478	4.0	No	South	Other	Monthly	
998	64.0	Male	22.096111	2.0	No	West	White Collar	Rarely	
999	66.0	Female	NaN	2.0	No	South	Blue Collar	Rarely	

10 rows × 35 columns

Selecting Data:

Select specific rows and columns using square brackets ([]), loc[], or iloc[].

```
In [131]: # Creating a DataFrame from a dictionary
data1 = {'Name': ['Alice', 'Bob', 'Charlie', 'David'],
         'Age': [25, 30, 35, 40]}
df = pd.DataFrame(data1)
df
```

```
Out[131]:
```

	Name	Age
0	Alice	25
1	Bob	30
2	Charlie	35
3	David	40

```
In [135]: # Selecting a single column
df['Name']
```

```
Out[135]:
```

0	Alice
1	Bob
2	Charlie
3	David

Name: Name, dtype: object

```
In [136]: df
```

Out[136]:

	Name	Age
0	Alice	25
1	Bob	30
2	Charlie	35
3	David	40

In [134... *# Selecting rows by index label*
print(df.loc[0])

Name Alice
Age 25
Name: 0, dtype: object

In [163... df

Out[163]:

	Name	Age
0	Alice	25
1	Bob	30
2	Charlie	35
3	David	40

In [167... print(df.loc[[3,0,2]])

Name Age
3 David 40
0 Alice 25
2 Charlie 35

In [162... *# Selecting rows by index label*
print(df.loc[0]) *#row*

Name Alice
Age 25
Name: 0, dtype: object

In [168... df

Out[168]:

	Name	Age
0	Alice	25
1	Bob	30
2	Charlie	35
3	David	40

In [172... *# Selecting rows and columns by index position*
print(df.iloc[2, 1]) *#row=2(charlie),col=1(age)*

#name=0
#age=1

35

```
In [161... # Selecting rows and columns by index position
print(df.iloc[1:4, 0:2]) #0,1 row, 1 column
```

	Name	Age
1	Bob	30
2	Charlie	35
3	David	40

```
In [155... df
```

```
Out[155]:
```

	Name	Age
0	Alice	25
1	Bob	30
2	Charlie	35
3	David	40

```
In [26]: # Selecting rows and columns by index position
print(df.iloc[0:3, 0:2])
```

	Name	Age
0	Alice	25
1	Bob	30
2	Charlie	35

?? To obtain the Age, Sex, and BMI columns, along with the first 10 rows of the DataFrame,

```
In [30]: print(data.iloc[0:11, 0:3])
```

	Age	Sex	BMI
0	56.0	Male	26.787631
1	69.0	Male	22.063653
2	46.0	Female	24.036101
3	32.0	Male	24.229816
4	60.0	Male	31.784284
5	25.0	Female	23.202814
6	78.0	Male	31.783167
7	38.0	Male	11.437548
8	56.0	Male	31.739279
9	75.0	Male	NaN
10	36.0	Male	29.511273

Indexing Method	Loc Method	Iloc Method
How to select data	Labels/ values of object	Position of object
Technique	name-based	position-based
Ordering	Name-based ordering	Position-based ordering
Sorting	Based on category	Smallest to largest

Cleaning Data:

Handle missing data using methods like `dropna()`, `fillna()`, or `interpolate()`.

Remove duplicate rows with `drop_duplicates()`.

```
In [179... data=pd.read_excel("C:/Users/HP/Desktop/datasets/mydata.xlsx")
```

```
In [77]: data
```

```
Out[77]:
```

	roll no	first name	last name	mark	gender
0	1	ammu	sl	12.0	female
1	1	ammu	sl	12.0	female
2	3	appu	kumar	15.0	male
3	4	appu	sha	20.0	male
4	5	malu	ja	NaN	female
5	6	saju	sa	16.0	NaN

```
In [78]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   roll no     6 non-null     int64
1   first name  6 non-null     object
2   last name   6 non-null     object
3   mark        5 non-null     float64
4   gender      5 non-null     object
dtypes: float64(1), int64(1), object(3)
memory usage: 368.0+ bytes
```

```
In [180]: data.isnull()
```

```
Out[180]:
```

	roll no	first name	last name	mark	gender
0	False	False	False	False	False
1	False	False	False	False	False
2	False	False	False	False	False
3	False	False	False	False	False
4	False	False	False	True	False
5	False	False	False	False	True

```
In [80]: data.isnull().sum()
```

```
Out[80]:
```

roll no	0
first name	0
last name	0
mark	1
gender	1
dtype: int64	

```
In [182]: newdata=data.dropna()#not available  
newdata
```

```
Out[182]:
```

	roll no	first name	last name	mark	gender
0	1	ammu	sl	12.0	female
1	1	ammu	sl	12.0	female
2	3	appu	kumar	15.0	male
3	4	appu	sha	20.0	male

```
In [82]: data
```

```
Out[82]:
```

	roll no	first name	last name	mark	gender
0	1	ammu	sl	12.0	female
1	1	ammu	sl	12.0	female
2	3	appu	kumar	15.0	male
3	4	appu	sha	20.0	male
4	5	malu	ja	NaN	female
5	6	saju	sa	16.0	NaN

```
In [187]: data.fillna(0)
```

```
Out[187]:
```

	roll no	first name	last name	mark	gender
0	1	ammu	sl	12.0	female
1	1	ammu	sl	12.0	female
2	3	appu	kumar	15.0	male
3	4	appu	sha	20.0	male
4	5	malu	ja	0.0	female
5	6	saju	sa	16.0	0

```
In [84]: data.duplicated()
```

```
Out[84]:
0    False
1     True
2    False
3    False
4    False
5    False
dtype: bool
```

```
In [188... data.drop_duplicates()
```

```
Out[188]:
```

	roll no	first name	last name	mark	gender
0	1	ammu	sl	12.0	female
2	3	appu	kumar	15.0	male
3	4	appu	sha	20.0	male
4	5	malu	ja	NaN	female
5	6	saju	sa	16.0	NaN

```
In [88]: #we can clean the data
```

Data Manipulation:

```
In [193... data["full name "]#no column
```

```

-----
KeyError                                Traceback (most recent call last)
~\anaconda3\lib\site-packages\pandas\core\indexes\base.py in get_loc(self, key, metho
d, tolerance)
    3628             try:
-> 3629                 return self._engine.get_loc(casted_key)
    3630             except KeyError as err:

~\anaconda3\lib\site-packages\pandas\_libs\index.pyx in pandas._libs.index.IndexEngin
e.get_loc()

~\anaconda3\lib\site-packages\pandas\_libs\index.pyx in pandas._libs.index.IndexEngin
e.get_loc()

pandas\_libs\hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.g
et_item()

pandas\_libs\hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.g
et_item()

KeyError: 'full name '

```

The above exception was the direct cause of the following exception:

```

KeyError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_5824\2332282219.py in <module>
----> 1 data["full name "]#no column

~\anaconda3\lib\site-packages\pandas\core\frame.py in __getitem__(self, key)
    3503         if self.columns.nlevels > 1:
    3504             return self._getitem_multilevel(key)
-> 3505         indexer = self.columns.get_loc(key)
    3506         if is_integer(indexer):
    3507             indexer = [indexer]

~\anaconda3\lib\site-packages\pandas\core\indexes\base.py in get_loc(self, key, metho
d, tolerance)
    3629             return self._engine.get_loc(casted_key)
    3630             except KeyError as err:
-> 3631                 raise KeyError(key) from err
    3632             except TypeError:
    3633                 # If we have a listlike key, _check_indexing_error will raise

KeyError: 'full name '

```

```
In [202... data["full name"]=data["first name "] + "    " +data["last name"]
```

```
In [203... data["full name"]
```

```

Out[203]:
0      ammu    sl
1      ammu    sl
2      appu    kumar
3      appu    sha
4      malu    ja
5      saju    sa
Name: full name, dtype: object

```

```
In [204... data
```

Out[204]:

	roll no	first name	last name	mark	gender	fullname	full name
0	1	ammu	sl	12.0	female	ammusl	ammu sl
1	1	ammu	sl	12.0	female	ammusl	ammu sl
2	3	appu	kumar	15.0	male	appukumar	appu kumar
3	4	appu	sha	20.0	male	appusha	appu sha
4	5	malu	ja	NaN	female	maluja	malu ja
5	6	saju	sa	16.0	NaN	sajusa	saju sa

In []: